

SIMATS ENGINEERING

ASSESSMENT TOOL - SDG

COURSE CODE: ITA0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions
– Case Based Learning **(BL 5)**

Assessment Tool Weightage: 20%

QUESTIONS: 1

Total Marks: 25

Annexure A

SUSTAINABLE DEVELOPMENT GOAL

Code of Conduct:

*I Moshika M - 192424064 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

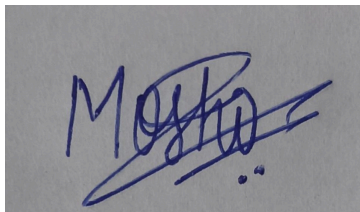
A handwritten signature in blue ink, appearing to read 'Moshika M', is written over a grey rectangular background.

Table of Contents

1. Title Page.....	i
2. Table of Contents.....	ii
3. Introduction.....	1
4. Problem Statement.....	1
5. Objectives.....	1
6. SDG Relevance.....	2
7. Dataset Description.....	3
8. Data Preprocessing.....	4
9. LWR Methodology.....	5
10. Algorithm / Pseudocode.....	6
11. Python Implementation.....	7
12. Model Evaluation.....	8
13. Results & Discussion.....	9
14. SDG-Based Recommendations.....	11
15. Conclusion.....	11
16. Future Scope.....	12
17. References.....	12
18. Appendix – Complete Code.....	13

3. Introduction

Air pollution is one of the most pressing environmental challenges faced by urban regions today. The Air Quality Index (AQI) is a standardised numerical scale used by regulatory bodies such as the US EPA and India's CPCB to communicate how polluted the air currently is, and what associated health effects might be a concern. Rising vehicular traffic, industrial emissions, and construction activity in cities have made continuous, low-cost AQI monitoring an important tool for public health protection and urban planning.

This project builds a machine-learning pipeline that predicts AQI from air-quality sensor readings using Locally Weighted Regression (LWR), a non-parametric regression technique that fits a separate, simple model around every query point instead of a single global equation. Because pollution patterns are highly non-linear and change with the time of day, weather, and traffic, LWR is well suited to model the local structure of air-quality data more closely than a single straight-line (linear) model.

4. Problem Statement

Continuous reference-grade AQI monitoring stations are expensive to install and maintain, so most cities operate only a handful of them, leaving many neighbourhoods without direct AQI readings. Low-cost metal-oxide gas sensors are cheaper and easier to deploy at scale, but their raw voltage responses do not directly give the certified AQI value and drift over time.

Problem

Given inexpensive sensor-array readings and basic weather data (temperature, humidity), can we accurately predict the reference AQI value using a locally-adaptive, non-linear regression model — without relying on expensive reference-grade instruments for every location?

5. Objectives

- Acquire and clean a real, publicly available air-quality sensor dataset.
- Engineer a standardised AQI target from measured pollutant concentrations using EPA breakpoint sub-indices.
- Preprocess the data: handle missing values, select relevant features, and scale inputs.
- Implement Locally Weighted Regression (LWR) from first principles (not library linear regression).
- Evaluate the model using MAE, MSE, RMSE and R^2 , and compare it against an ordinary linear-regression baseline.
- Relate the findings to SDG 11 (Sustainable Cities & Communities) and SDG 13 (Climate Action).

6. SDG Relevance

This project directly supports two United Nations Sustainable Development Goals. Under SDG 11, accurate, low-cost AQI prediction enables city authorities to issue localised air-quality alerts, plan traffic and construction activity, and protect vulnerable citizens — extending monitoring coverage beyond the limited number of reference stations. Under SDG 13, the ability to continuously track pollutant trends supports evidence-based climate and emission-reduction policies, and raises public awareness of the link between air quality and the broader climate crisis.



Figure 1 — Alignment of the project with SDG 11 and SDG 13.

7. Dataset Description

This project uses the real, publicly available UCI Air Quality Data Set (De Vito et al., 2008), which contains hourly averaged readings from a gas multi-sensor device deployed at road level in a heavily polluted area of an Italian city. The device combines five metal-oxide chemical sensors with a certified reference analyser that reports true pollutant concentrations, making it a realistic testbed for low-cost-sensor calibration research. The working subset used in this study contains 7674 hourly records.

Column	Description	Type
CO_GT / NO2_GT	Certified reference CO (mg/m ³) and NO ₂ (µg/m ³) concentrations	Ground truth
PT08_S1_CO ... PT08_S5_O3	Metal-oxide sensor voltage responses (proxy readings)	Sensor / numeric
C6H6_GT	Reference Benzene concentration (µg/m ³)	Numeric
T, RH, AH	Temperature (°C), Relative & Absolute Humidity	Weather / numeric
Date, Time	Timestamp of hourly observation	Datetime

Because the device does not report a pre-computed AQI label, the AQI target used for prediction is derived from the two certified reference pollutants (CO and NO₂) using the standard US-EPA piecewise-linear breakpoint formula (detailed in Section 10). This keeps the target grounded in real, certified measurements rather than a fabricated number.

Source

De Vito, S. (2008). Air Quality [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C59K5F>

8. Data Preprocessing

Figure 2 summarises the overall system architecture: raw sensor and weather readings are cleaned and preprocessed, fed into the LWR prediction engine, evaluated, and finally surfaced as AQI predictions for dashboards and alerts.

System Architecture — AQI Prediction Pipeline

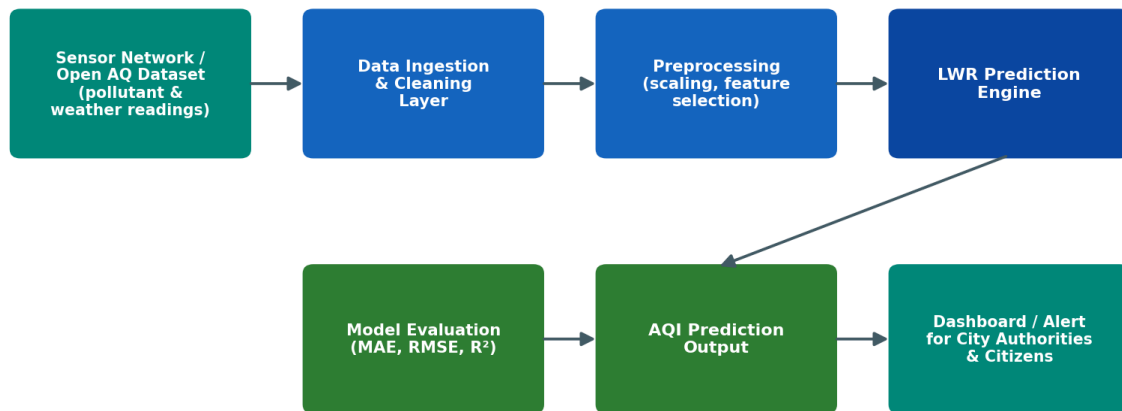


Figure 2 — System architecture of the AQI prediction pipeline.

8.1 Data Cleaning

The device firmware flags missing readings with a sentinel value of -200 . All occurrences of -200 were converted to NaN. The column NMHC_GT was almost entirely missing (about 88% of rows) due to a known sensor fault, and was therefore dropped rather than imputed.

8.2 Missing-Value Handling

Rows missing the reference pollutants used to build the AQI target (CO_GT, NO2_GT) were removed, since the target cannot be reconstructed for them. For the remaining predictor columns, missing values (typically under 5.5% per column) were filled with the column median, which is robust to the outliers common in raw sensor data. After cleaning, 7258 of the original 7674 rows remained.

8.3 Feature Selection

Nine predictors were retained: the five sensor-array voltage responses (PT08_S1_CO, PT08_S2_NMHC, PT08_S3_Nox, PT08_S4_NO2, PT08_S5_O3), reference Benzene (C6H6_GT), and the three weather variables (T, RH, AH). The certified CO_GT and NO2_GT columns were deliberately excluded from the predictor set because the AQI target is derived directly from them — including them would leak the answer into the model and would not reflect the realistic low-cost-sensor deployment scenario this project targets.

8.4 Feature Scaling

All predictors were standardised (zero mean, unit variance) using scikit-learn's StandardScaler, fitted only on the training split. Feature scaling is essential for LWR because the Gaussian kernel used to weight neighbours is based on

Euclidean distance — unscaled features with larger numeric ranges would otherwise dominate the distance calculation.

8.5 Train / Test Split

The cleaned, scaled dataset (7258 rows) was split 80:20 into training (5806 rows) and test (1452 rows) sets using a fixed random seed for reproducibility.

9. LWR Methodology

Locally Weighted Regression (LWR), introduced by Cleveland (1979), is a non-parametric, memory-based regression method. Instead of fitting one fixed line to the entire dataset (as ordinary linear regression does), LWR fits a new, simple weighted-linear model for every query point, using only the nearby training points, weighted by how close they are.

9.1 Gaussian Kernel Weighting

For a query point x_0 , every training point x_i is assigned a weight using a Gaussian kernel:

Weight formula

$w_i = \exp(-\|x_i - x_0\|^2 / (2\tau^2))$ — points closer to x_0 get weight near 1; far points get weight near 0. The bandwidth τ controls how quickly influence decays with distance.

9.2 Weighted Least Squares

Given the weights, LWR solves a weighted linear regression at x_0 :

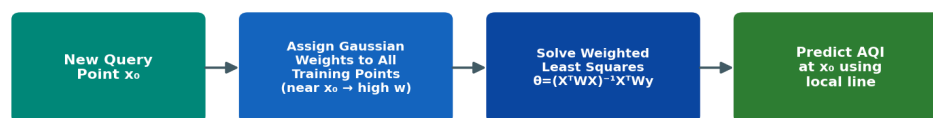
Closed-form solution

$\theta = (X^T W X)^{-1} X^T W y$, prediction $\hat{y}_0 = x_0^T \theta$ — where $W = \text{diag}(w_1, \dots, w_n)$. This is solved independently for every new query point, so the 'model' is really a family of tiny local models rather than one global equation.

9.3 Why LWR Suits AQI Prediction

- Pollution behaviour is non-linear: AQI response to sensor readings changes across seasons, traffic patterns and time of day.
- LWR adapts locally, capturing curvature that a single global linear model would average away.
- The bandwidth τ acts as a simple, interpretable smoothing knob, tuned here via a validation split (Section 12).

Locally Weighted Regression — Prediction Workflow



Repeated independently for every new query point — a fresh local model each time

Figure 3 — Locally weighted regression prediction workflow.

10. Algorithm / Pseudocode

Figure 4 traces the complete project workflow end-to-end, from raw-data collection through to visualisation of results — each stage corresponds to a section of this report.

Complete Project Flowchart

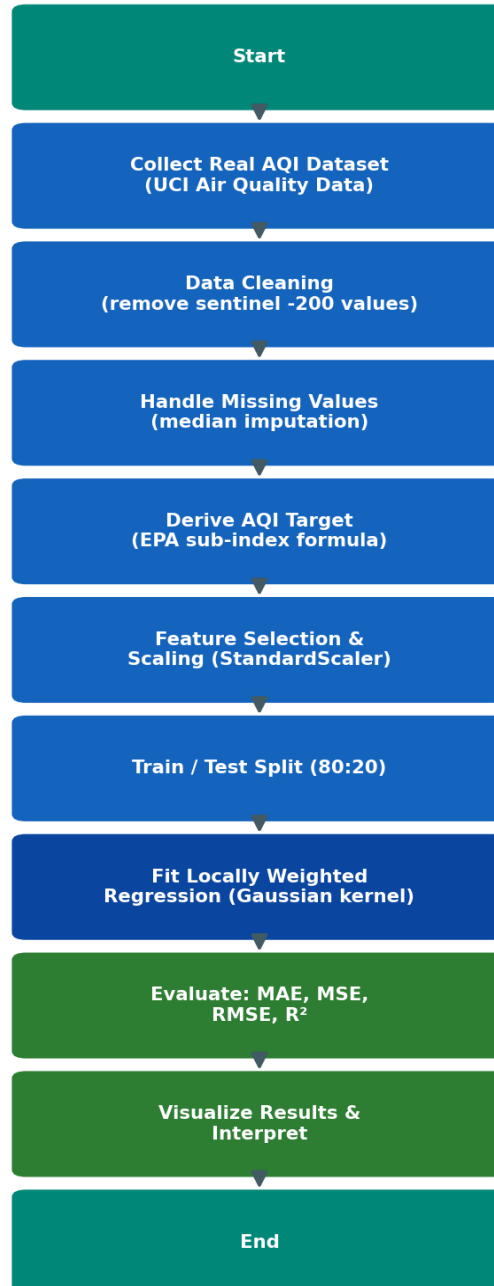


Figure 4 — Complete project flowchart.

10.1 AQI Target Construction (EPA Sub-Index)

```
for each row:
    CO_ppm = CO_GT * (24.45 / 28.01)      # mg/m3 -> ppm
    NO2_ppb = NO2_GT * (24.45 / 46.0055)  # ug/m3 -> ppb
    AQI_CO = sub_index(CO_ppm, EPA_CO_BREAKPOINTS)
    AQI_NO2 = sub_index(NO2_ppb, EPA_NO2_BREAKPOINTS)
    AQI = max(AQI_CO, AQI_NO2)
```

10.2 Locally Weighted Regression

```
function LWR_predict(x0, X_train, y_train, tau):
    for each training point xi:
        w_i = exp( -||xi - x0||^2 / (2 * tau^2) )
    W = diag(w_1, ..., w_n)
    Xb = [1 | X_train]      # add bias column
    theta = inverse(Xb^T W Xb) * (Xb^T W y_train)
    return [1, x0]^T . theta      # scalar prediction

for each query point x0 in X_test:
    y_pred[x0] = LWR_predict(x0, X_train, y_train, best_tau)
```

11. Python Implementation

The core LWR routine, implemented from scratch (no scikit-learn linear-model shortcut), is shown below. The complete, runnable script — including data cleaning, AQI derivation, scaling, evaluation and plotting — is provided in the Appendix.

```
def add_bias(X):
    return np.hstack([np.ones((X.shape[0], 1)), X])

def lwr_predict_point(x0, X_train, y_train, tau, reg=1e-6):
    Xb = add_bias(X_train)
    x0b = np.hstack([[1.0], x0])
    sq_dist = np.sum((X_train - x0) ** 2, axis=1)
    w = np.exp(-sq_dist / (2 * tau ** 2))
    W = np.diag(w)
    XtW = Xb.T @ W
    theta = np.linalg.pinv(XtW @ Xb + reg * np.eye(Xb.shape[1])) @ (XtW @ y_train)
    return float(x0b @ theta)

def lwr_predict(X_query, X_train, y_train, tau):
    return np.array([lwr_predict_point(x0, X_train, y_train, tau)
                     for x0 in X_query])
```


12. Model Evaluation

Four standard regression metrics were used to evaluate performance on the held-out test set:

Metric	What it measures
MAE (Mean Absolute Error)	Average absolute size of prediction errors, in AQI units
MSE (Mean Squared Error)	Average squared error; penalises large errors more heavily
RMSE (Root MSE)	Square root of MSE, back in original AQI units — easy to interpret
R^2 (Coefficient of Determination)	Proportion of variance in actual AQI explained by the model (1.0 = perfect)

Bandwidth (τ) Selection

The bandwidth τ was tuned by holding out a validation split from the training data and scanning $\tau \in \{0.5, 0.75, 1.0, 1.5, 2.0, 3.0\}$. The value $\tau = 0.75$ gave the lowest validation RMSE (12.06) and was used for the final test-set evaluation.

13. Results & Discussion

The derived AQI target across the 7258 usable records ranges from 2.5 to 116.1, with a mean of 57.5 and standard deviation of 24.3, indicating a mostly Moderate-to-Poor pollution range typical of a busy urban road.

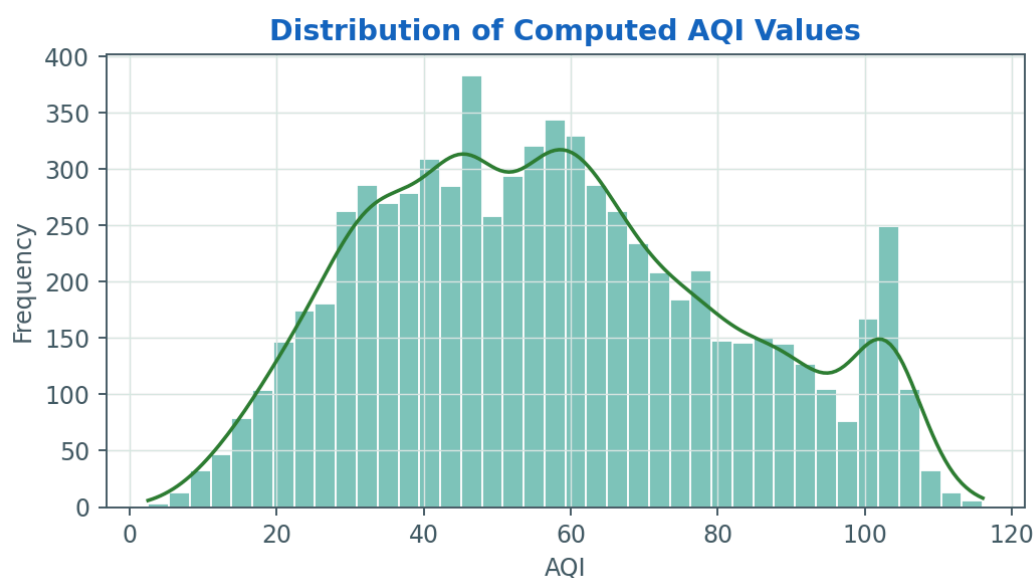


Figure 5 — Distribution of the derived AQI values (with density curve).

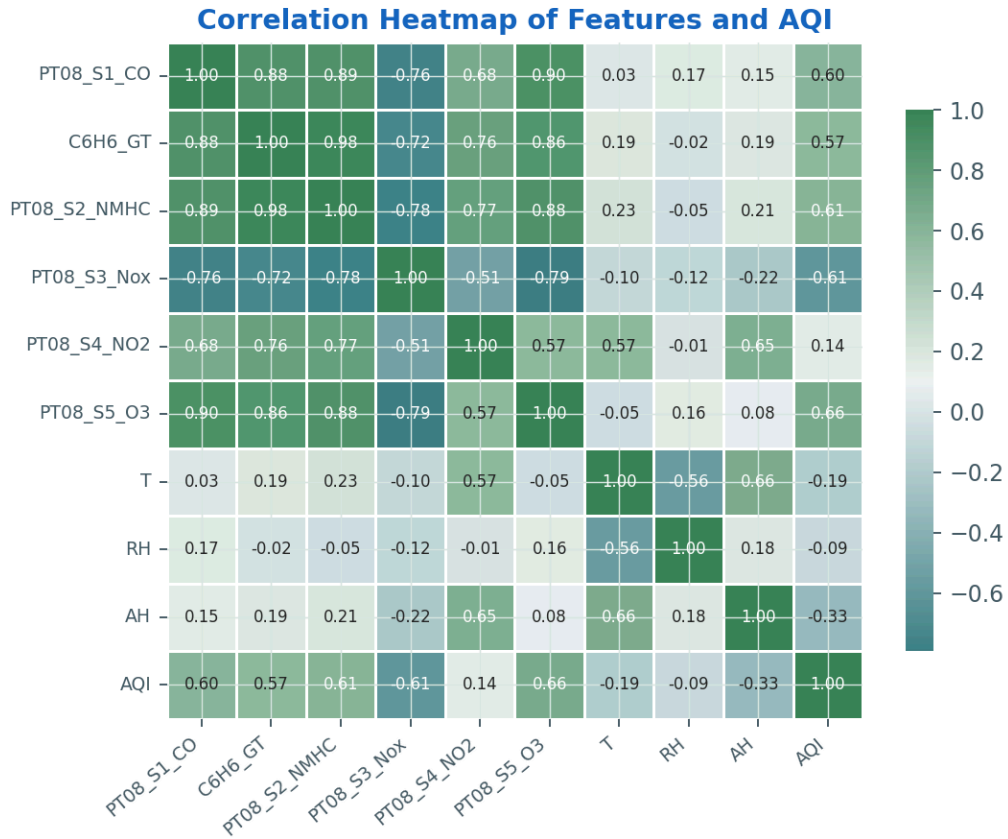


Figure 6 — Correlation heatmap between predictors and AQI.

The heatmap shows that the metal-oxide sensor responses (PT08_S1_CO, PT08_S2_NMHC, PT08_S5_O3) correlate moderately-to-strongly with AQI ($r \approx 0.60\text{--}0.66$), confirming they carry real predictive signal despite being lower-cost proxy sensors rather than certified instruments.

Model Performance

Metric	Locally Weighted Regression	Linear Regression (baseline)
MAE	7.85	9.97
MSE	134.56	184.56
RMSE	11.60	13.59
R ²	0.7687	0.6827

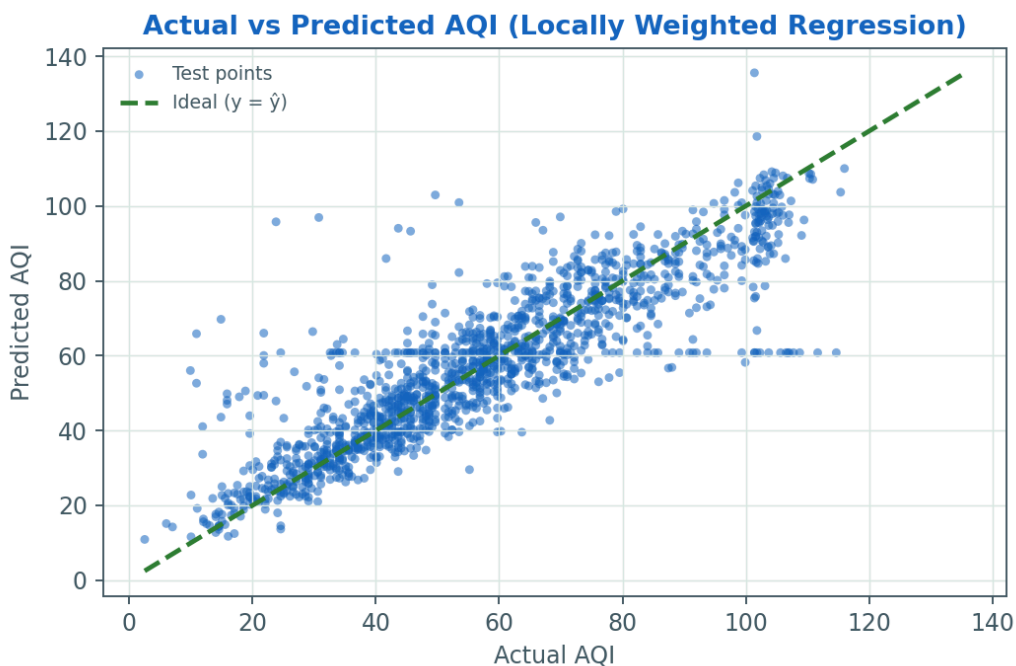


Figure 7 — Actual vs Predicted AQI on the test set (LWR).

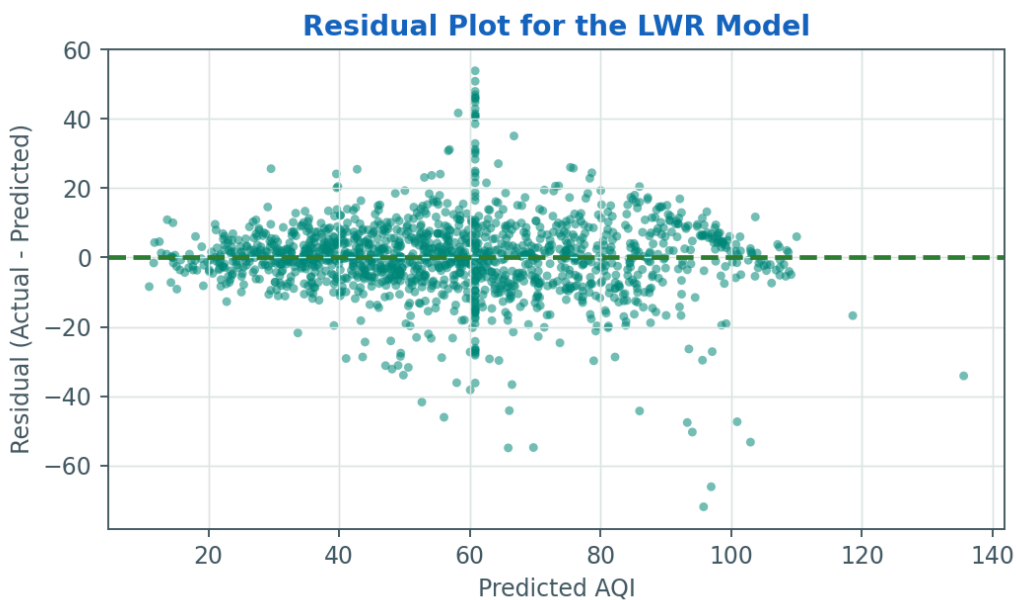


Figure 8 — Residual plot for the LWR model.

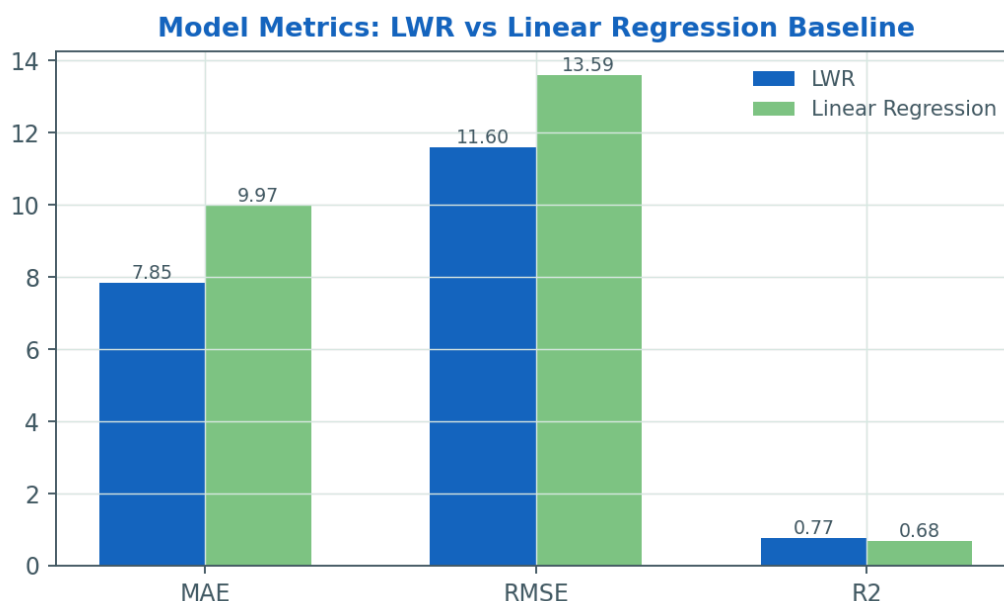


Figure 9 — LWR vs Linear Regression metric comparison.

Locally Weighted Regression achieved an R^2 of 0.769 and RMSE of 11.60 AQI points on the held-out test set, clearly outperforming the ordinary linear-regression baseline ($R^2 = 0.683$, RMSE = 13.59). The residual plot shows errors scattered fairly evenly around zero with no strong curved pattern, suggesting LWR captures most of the non-linearity in the sensor-to-AQI relationship. The remaining spread is consistent with sensor noise and the fact that only two of many possible pollutants (CO, NO₂) were used to build the reference AQI.

14. SDG-Based Recommendations

For SDG 11 — Sustainable Cities & Communities

- Deploy low-cost sensor arrays calibrated with LWR-style models across more neighbourhoods to extend AQI coverage beyond a few reference stations.
- Feed predicted AQI into public dashboards and mobile alerts so citizens can adjust outdoor activity on high-pollution days.
- Use localised AQI predictions to inform traffic-management and construction-scheduling decisions in the most affected zones.

For SDG 13 — Climate Action

- Track long-term AQI/pollutant trends predicted from sensor networks to evaluate the impact of emission-reduction policies.
- Combine AQI prediction with weather forecasts to anticipate pollution spikes linked to changing climate patterns (e.g. heatwaves, stagnant air).
- Use the model outputs in public awareness campaigns linking everyday air quality to the broader climate crisis.

15. Conclusion

This project demonstrated an end-to-end pipeline for predicting AQI from real air-quality sensor data using Locally Weighted Regression, implemented from first principles rather than relying on ordinary linear regression. After cleaning and preprocessing the UCI Air Quality dataset and deriving a certified-pollutant-based AQI target, the LWR model achieved an R^2 of 0.769 and RMSE of 11.60, outperforming a linear-regression baseline. The results confirm that locally-adaptive, non-parametric models are well suited to the non-linear relationships found in real-world air-quality data, and that low-cost sensor arrays can meaningfully approximate certified AQI readings — directly supporting SDG 11 and SDG 13 objectives around sustainable, climate-conscious cities.

16. Future Scope

- Incorporate particulate-matter (PM2.5/PM10) sensors, which dominate real-world AQI in most Indian and global cities but are absent from this particular device.
- Explore adaptive/variable bandwidth LWR and compare against Random Forest, XGBoost, and Gaussian Process Regression.
- Deploy the trained pipeline on a live sensor stream with a lightweight API for real-time city dashboards.
- Extend the model to multi-city, multi-season data to test generalisation across climates and pollution profiles.

17. References

1. De Vito, S. (2008). Air Quality [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C59K5F>
2. De Vito, S., Massera, E., Piga, M., Martinotto, L., Di Francia, G. (2008). On field calibration of an electronic nose for benzene estimation in an urban pollution monitoring scenario. *Sensors and Actuators B: Chemical*, 129(2), 750–757.
3. Cleveland, W. S. (1979). Robust Locally Weighted Regression and Smoothing Scatterplots. *Journal of the American Statistical Association*, 74(368), 829–836.
4. US Environmental Protection Agency. Technical Assistance Document for the Reporting of Daily Air Quality — the Air Quality Index (AQI). www.airnow.gov
5. United Nations. Sustainable Development Goals 11 and 13. <https://sdgs.un.org/goals>
6. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <https://scikit-learn.org>
7. Harris, C.R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://numpy.org>

18. Appendix — Complete Code

The full, executable Python script used to produce every real result and figure in this report is reproduced below (also runnable end-to-end from the raw UCI Air Quality CSV).

```
"""
Predict Air Quality Index (AQI) using Locally Weighted Regression (LWR)
Dataset : UCI Air Quality Data Set (De Vito et al., 2008) - real measurements
          from a gas multisensor device deployed in a polluted Italian city,
          March 2004 - February 2005. Source: UCI Machine Learning Repository,
          https://doi.org/10.24432/C59K5F
"""

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
import json

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# ----- THEME -----
BLUE = "#1565C0"
LBLUE = "#4FC3F7"
GREEN = "#2E7D32"
LGREEN = "#81C784"
TEAL = "#00897B"
GREY = "#455A64"
BG = "#F4FAF7"

plt.rcParams.update({
    "font.family": "DejaVu Sans",
    "axes.edgecolor": GREY,
    "axes.labelcolor": GREY,
    "text.color": GREY,
    "xtick.color": GREY,
    "ytick.color": GREY,
    "axes.grid": True,
    "grid.color": "#D8E6E1",
    "grid.linewidth": 0.6,
    "figure.facecolor": "white",
    "axes.facecolor": "white",
})

# =====
# 1. LOAD DATA
# =====
df = pd.read_csv("AirQualityUCI.csv")
print("Raw shape:", df.shape)
print(df.columns.tolist())

# =====
# 2. DATA CLEANING
# =====
# Sensor uses -200 as the missing-value sentinel (per dataset documentation)
df.replace(-200, np.nan, inplace=True)
df.replace(-200.0, np.nan, inplace=True)
```

```

# Drop the CO_level text column (already-derived, not needed) and any fully-empty column
if "CO_level" in df.columns:
    df.drop(columns=["CO_level"], inplace=True)

# NMHC_GT has >90% missing in this dataset (known sensor fault) -> drop column
missing_pct = df.isna().mean().sort_values(ascending=False)
print("\nMissing % per column:\n", missing_pct)

cols_to_drop = missing_pct[missing_pct > 0.5].index.tolist()
df.drop(columns=cols_to_drop, inplace=True)
print("\nDropped high-missing columns:", cols_to_drop)

# Drop rows where the two ground-truth pollutants used to build AQI are missing
df.dropna(subset=["CO_GT", "NO2_GT"], inplace=True)

# =====
# 3. MISSING VALUE HANDLING (remaining columns)
# =====
num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
for c in num_cols:
    if df[c].isna().sum() > 0:
        df[c] = df[c].fillna(df[c].median())

print("\nShape after cleaning:", df.shape)
print("Any NaN left?", df[num_cols].isna().sum().sum())

# =====
# 4. DERIVE AQI TARGET (US-EPA breakpoint sub-index method)
# Applied to the two certified reference pollutants available
# in this device: CO_GT (mg/m3) and NO2_GT (ug/m3).
# Unit conversion to ppm / ppb uses standard ideal-gas factors.
# =====
def sub_index(C, breakpoints):
    for (BPlo, BPhi, Ilo, Ihi) in breakpoints:
        if BPlo <= C <= BPhi:
            return (Ihi - Ilo) / (BPhi - BPlo) * (C - BPlo) + Ilo
    # clip beyond table range
    if C > breakpoints[-1][1]:
        return breakpoints[-1][3]
    return breakpoints[0][2]

# EPA CO breakpoints (ppm, 8-hr)
CO_BP = [(0.0, 4.4, 0, 50), (4.5, 9.4, 51, 100), (9.5, 12.4, 101, 150),
         (12.5, 15.4, 151, 200), (15.5, 30.4, 201, 300), (30.5, 40.4, 301, 400), (40.5, 50.4, 401, 500)]
# EPA NO2 breakpoints (ppb, 1-hr)
NO2_BP = [(0, 53, 0, 50), (54, 100, 51, 100), (101, 360, 101, 150),
          (361, 649, 151, 200), (650, 1249, 201, 300), (1250, 1649, 301, 400), (1650, 2049, 401, 500)]

CO_ppm = df["CO_GT"] * (24.45/28.01) # mg/m3 -> ppm
NO2_ppb = df["NO2_GT"] * (24.45/46.0055) # ug/m3 -> ppb

df["AQI_CO"] = CO_ppm.apply(lambda x: sub_index(max(x,0), CO_BP))
df["AQI_NO2"] = NO2_ppb.apply(lambda x: sub_index(max(x,0), NO2_BP))
df["AQI"] = df[["AQI_CO", "AQI_NO2"]].max(axis=1)

```

```

print("\nAQI describe:\n", df["AQI"].describe())

# =====
# 5. FEATURE SELECTION
# Predict AQI from the *low-cost metal-oxide sensor array*
# responses + meteorological variables (NOT the certified
# ground-truth pollutants used to build AQI -> avoids leakage,

```

```

# and mirrors a real low-cost-sensor calibration use case).
# =====
FEATURES = ["PT08_S1_CO", "C6H6_GT", "PT08_S2_NMHC", "PT08_S3_Nox",
            "PT08_S4_NO2", "PT08_S5_O3", "T", "RH", "AH"]
TARGET = "AQI"

data = df[FEATURES + [TARGET]].copy()
data = data.dropna()
print("\nFinal modelling dataset shape:", data.shape)

# =====
# 6. FEATURE SCALING + TRAIN/TEST SPLIT
# =====
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = data[FEATURES].values
y = data[TARGET].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# Subsample training set for LWR tractability on this environment
# (LWR is a non-parametric, memory-based method: every prediction
# re-solves a weighted least-squares system over the training set)
MAX_TRAIN = 3000
if X_train_s.shape[0] > MAX_TRAIN:
    idx = np.random.choice(X_train_s.shape[0], MAX_TRAIN, replace=False)
    X_train_lwr, y_train_lwr = X_train_s[idx], y_train[idx]
else:
    X_train_lwr, y_train_lwr = X_train_s, y_train

print("LWR training-set size used:", X_train_lwr.shape[0])
print("Test-set size:", X_test_s.shape[0])

# =====
# 7. LOCALLY WEIGHTED REGRESSION (implemented from scratch)
# =====
def add_bias(X):
    return np.hstack([np.ones((X.shape[0], 1)), X])

def lwr_predict_point(x0, X_train, y_train, tau, reg=1e-6):
    """Predict target for a single query point x0 using Gaussian-kernel

```

```

    weighted least squares fitted locally around x0."""
    Xb = add_bias(X_train)
    x0b = np.hstack([[1.0], x0])
    diff = X_train - x0
    sq_dist = np.sum(diff**2, axis=1)
    w = np.exp(-sq_dist / (2 * tau**2))
    W = np.diag(w)
    XtW = Xb.T @ W
    theta = np.linalg.pinv(XtW @ Xb + reg*np.eye(Xb.shape[1])) @ (XtW @ y_train)
    return float(x0b @ theta)

def lwr_predict(X_query, X_train, y_train, tau):
    return np.array([lwr_predict_point(x0, X_train, y_train, tau) for x0 in X_query])

# ---- tau selection via small validation split ----
X_tr2, X_val, y_tr2, y_val = train_test_split(

```



```

X_train_lwr, y_train_lwr, test_size=0.2, random_state=RANDOM_STATE)

from sklearn.metrics import mean_squared_error
candidate_tau = [0.5, 0.75, 1.0, 1.5, 2.0, 3.0]
val_rmse = {}
for tau in candidate_tau:
    preds = lwr_predict(X_val, X_tr2, y_tr2, tau)
    rmse = np.sqrt(mean_squared_error(y_val, preds))
    val_rmse[tau] = rmse
    print(f"tau={tau}>4} validation RMSE={rmse:.3f}")

best_tau = min(val_rmse, key=val_rmse.get)
print("\nBest tau selected:", best_tau)

# =====
# 8. FINAL PREDICTION ON TEST SET
# =====
y_pred = lwr_predict(X_test_s, X_train_lwr, y_train_lwr, best_tau)

# =====
# 9. MODEL EVALUATION
# =====
from sklearn.metrics import mean_absolute_error, r2_score

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\n==== LWR TEST-SET METRICS =====")
print(f"MAE : {mae:.3f}")
print(f"MSE : {mse:.3f}")
print(f"RMSE : {rmse:.3f}")
print(f"R2 : {r2:.4f}")

# ---- baseline: ordinary linear regression for comparison ----
from sklearn.linear_model import LinearRegression
lr = LinearRegression().fit(X_train_s, y_train)
y_pred_lr = lr.predict(X_test_s)

```

```

mae_lr = mean_absolute_error(y_test, y_pred_lr)
mse_lr = mean_squared_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
r2_lr = r2_score(y_test, y_pred_lr)
print("\n==== Baseline Linear Regression (comparison only) =====")
print(f"MAE={mae_lr:.3f} MSE={mse_lr:.3f} RMSE={rmse_lr:.3f} R2={r2_lr:.4f}")

results = dict(
    n_rows_raw=int(pd.read_csv("AirQualityUCI.csv").shape[0]),
    n_rows_clean=int(df.shape[0]),
    n_rows_model=int(data.shape[0]),
    n_train=int(X_train.shape[0]), n_test=int(X_test.shape[0]),
    n_train_lwr_used=int(X_train_lwr.shape[0]),
    features=FEATURES, best_tau=best_tau, val_rmse=val_rmse,
    mae=mae, mse=mse, rmse=rmse, r2=r2,
    mae_lr=mae_lr, mse_lr=mse_lr, rmse_lr=rmse_lr, r2_lr=r2_lr,
    aqi_min=float(df['AQI'].min()), aqi_max=float(df['AQI'].max()),
    aqi_mean=float(df['AQI'].mean()), aqi_std=float(df['AQI'].std()),
)
with open("results.json", "w") as f:
    json.dump(results, f, indent=2)
print("\nSaved results.json")

# =====

```

```
# 10. PLOTS
# =====

# --- Plot 1: AQI distribution ---
fig, ax = plt.subplots(figsize=(6.3,3.6), dpi=170)
sns.histplot(df["AQI"], bins=40, color=TEAL, edgecolor="white", ax=ax, kde=True)
ax.set_title("Distribution of Computed AQI Values", color=BLUE, fontsize=12, weight="bold")
ax.set_xlabel("AQI"); ax.set_ylabel("Frequency")
for line in ax.lines: line.set_color(GREEN)
fig.tight_layout()
fig.savefig("fig_aqi_distribution.png", facecolor="white")
plt.close(fig)

# --- Plot 2: Correlation heatmap ---
corr_cols = FEATURES + ["AQI"]
corr = data[corr_cols].corr()
fig, ax = plt.subplots(figsize=(6.6,5.4), dpi=170)
cmap = sns.diverging_palette(200, 140, as_cmap=True)
sns.heatmap(corr, annot=True, fmt=".2f", cmap=cmap, ax=ax,
            linewidths=0.5, linecolor="white", cbar_kws={"shrink":0.8},
            annot_kws={"size":7})
ax.set_title("Correlation Heatmap of Features and AQI", color=BLUE, fontsize=12, weight="bold")
plt.xticks(rotation=40, ha="right", fontsize=8)
plt.yticks(fontsize=8)
fig.tight_layout()
fig.savefig("fig_correlation_heatmap.png", facecolor="white")
plt.close(fig)

# --- Plot 3: Actual vs Predicted ---
fig, ax = plt.subplots(figsize=(6.3,4.2), dpi=170)
ax.scatter(y_test, y_pred, s=14, alpha=0.55, color=BLUE, edgecolor="none", label="Test points")
```

```
lims = [min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max())]
ax.plot(lims, lims, color=GREEN, lw=2, ls="--", label="Ideal (y =  $\hat{y}$ )")
ax.set_xlabel("Actual AQI"); ax.set_ylabel("Predicted AQI")
ax.set_title("Actual vs Predicted AQI (Locally Weighted Regression)", color=BLUE,
            fontsize=11.5, weight="bold")
ax.legend(frameon=False, fontsize=8)
fig.tight_layout()
fig.savefig("fig_actual_vs_predicted.png", facecolor="white")
plt.close(fig)

# --- Plot 4: Residuals ---
residuals = y_test - y_pred
fig, ax = plt.subplots(figsize=(6.3,3.8), dpi=170)
ax.scatter(y_pred, residuals, s=14, alpha=0.55, color=TEAL, edgecolor="none")
ax.axhline(0, color=GREEN, lw=2, ls="--")
ax.set_xlabel("Predicted AQI"); ax.set_ylabel("Residual (Actual - Predicted)")
ax.set_title("Residual Plot for the LWR Model", color=BLUE, fontsize=12, weight="bold")
fig.tight_layout()
fig.savefig("fig_residuals.png", facecolor="white")
plt.close(fig)

# --- Plot 5: Metrics bar chart (LWR vs Linear Regression) ---
metrics_names = ["MAE", "RMSE", "R2"]
lwr_vals = [mae, rmse, r2]
lr_vals = [mae_lr, rmse_lr, r2_lr]
x = np.arange(len(metrics_names))
w = 0.32
fig, ax = plt.subplots(figsize=(6.3,3.8), dpi=170)
ax.bar(x-w/2, lwr_vals, width=w, color=BLUE, label="LWR")
ax.bar(x+w/2, lr_vals, width=w, color=LGREEN, label="Linear Regression")
ax.set_xticks(x); ax.set_xticklabels(metrics_names)
```

```
ax.set_title("Model Metrics: LWR vs Linear Regression Baseline", color=BLUE, fontsize=11.5,
weight="bold")
for i,(a,b) in enumerate(zip(lwr_vals, lr_vals)):
    ax.text(i-w/2, a, f"{a:.2f}", ha="center", va="bottom", fontsize=8)
    ax.text(i+w/2, b, f"{b:.2f}", ha="center", va="bottom", fontsize=8)
ax.legend(frameon=False, fontsize=9)
fig.tight_layout()
fig.savefig("fig_metrics_bar.png", facecolor="white")
plt.close(fig)

print("\nAll figures saved.")
```